

FORMATION EMBARQUÉE

Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 1 — Fiche de séance 2 : OLED et architecture en modules (3 h)

En-tête pédagogique

Objectifs	Piloter un écran I2C ; découper un projet Arduino en modules <code>.h/.cpp</code> ; comprendre « interface vs implémentation » en pratique
Prérequis	Séance 1 finie et commitée ; module 02 §2 (classes) lu
Matériel	Montage de la séance 1 + OLED SSD1306 128×64 I2C
Livvable	Le même comportement qu'en séance 1, affiché sur OLED, avec un <code>.ino</code> < 60 lignes et 3 modules

Déroulé minuté

0:00-0:20 — Câblage OLED et scanner I2C

OLED : VCC → 5V (3,3 V selon module), GND → GND, SDA → A4, SCL → A5.

Avant toute bibliothèque d'écran, exécute un **scanner I2C** (croquis d'exemple « Wire → i2c_scanner » de l'IDE). Attendu : `0x3C` détecté. Méthode : *on valide le bus avant de déboguer l'écran* — si le scanner ne voit rien, aucune bibliothèque ne marchera (SDA/SCL inversés dans 80 % des cas).

0:20-0:50 — Premier affichage

Installe `Adafruit SSD1306` (+ `Adafruit GFX`). Mini-test isolé :

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(2);
  oled.setCursor(0, 0);
  oled.print(F("Bonjour"));
  oled.display();           // RIEN ne s'affiche sans display() !
}

void loop() {}
```

Piège n°1 de l'OLED : oublier `oled.display()` (tout se dessine dans un tampon en RAM, `display()` l'envoie à l'écran).

0:50-2:20 — LA refactorisation en modules (le cœur de la séance)

Objectif : le `.ino` ne doit plus contenir QUE l'orchestration. Crée trois paires de fichiers (IDE : bouton « ... » → Nouvel onglet) :

capteur.h — l'interface (ce que les autres ont le droit de savoir) :

```
#ifndef CAPTEUR_H
#define CAPTEUR_H
#include <stdint.h>

void capteur_init();
// À appeler à chaque tour : fait une mesure si l'intervalle est écoulé.
void capteur_tick(uint32_t maintenant_ms);
// Dernières valeurs valides (NAN si jamais rien reçu)
float capteur_temperature();
float capteur_humidite();
bool capteur_en_panne();           // 3 échecs consécutifs
#endif
```

capteur.cpp — l'implémentation (SEUL fichier qui connaît le DHT) :

```
#include "capteur.h"
#include <DHT.h>

static DHT dht(2, DHT22);           // static : invisible hors du module
static float t_ = NAN, h_ = NAN;
static uint8_t echecs_ = 0;
static uint32_t derniere_ = 0;

void capteur_init() { dht.begin(); }

void capteur_tick(uint32_t now) {
    if (now - derniere_ < 2000) return;
    derniere_ = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (isnan(t) || isnan(h)) {
        if (echecs_ < 255) echecs_++;
    } else {
        echecs_ = 0;
        t_ = t; h_ = h;
    }
}

float capteur_temperature() { return t_; }
float capteur_humidite()    { return h_; }
bool capteur_en_panne()     { return echecs_ >= 3; }
```

affichage.h/.cpp : même principe — `affichage_init()`, `affichage_tick(now, t, h, consigne, alerte, panne)` qui rafraîchit l'OLED au plus 5×/s. Personne d'autre n'inclut `Adafruit_SSD1306.h`.

alerte.h/.cpp : `alerte_tick(now, temperature, consigne)` qui rend l'état avec hystérésis et pilote la LED.

Le `.ino` final (c'est TOUT ce qu'il doit rester) :

```
#include "capteur.h"
#include "affichage.h"
#include "alerte.h"

void setup() {
    Serial.begin(115200);
    capteur_init();
    affichage_init();
    alerte_init();
}

void loop() {
    uint32_t now = millis();
    float consigne = 10.0f + analogRead(A0) * (30.0f / 1023.0f);

    capteur_tick(now);
    bool alarme = alerte_tick(now, capteur_temperature(), consigne);
    affichage_tick(now, capteur_temperature(), capteur_humidite(),
                  consigne, alarme, capteur_en_panne());
}
```

✓ **Point de contrôle (2:20)** — les trois critères d'architecture : 1. Le `.ino` fait < 60 lignes et n'inclut aucune bibliothèque matérielle. 2. `grep -r "Adafruit" *.ino` ne renvoie rien. 3. Question test : « pour remplacer l'OLED par un LCD 16×2, quels fichiers changent ? » — la réponse doit être : `affichage.cpp` (et lui seul).

2:20-2:50 — L'écran affiche l'état de panne

Utilise `capteur_en_panne()` pour afficher « CAPTEUR HS » en gros à l'écran (et rejoue le test du débranchement de la séance 1). Un système embarqué **montre** ses pannes — un écran qui affiche une vieille valeur sans le dire est un mensonge.

2:50-3:00 — Commit

```
git add . && git commit -m "TP1 seance 2 : OLED + architecture en modules"
```

Erreurs fréquentes

Symptôme	Cause	Remède
Écran blanc/noir mais scanner OK	oubli de <code>display()</code> , mauvaise taille (128×32 vs 64)	vérifier le constructeur
<code>multiple definition of...</code> à l'édition de liens	variable définie dans un <code>.h</code> inclus deux fois	définir dans le <code>.cpp</code> , <code>extern</code> ou accesseur dans le <code>.h</code>
Rien ne compile après découpage	garde d'inclusion oubliée, <code>#include</code> circulaire	<code>#ifndef/#define/#endif</code> partout
L'affichage « rame »	<code>display()</code> appelé à chaque tour de loop	le limiter à 5 Hz (tâche périodique)
RAM presque pleine à la compilation	chaînes sans <code>F()</code> , tampon OLED (1 Ko incompressible)	<code>F()</code> partout ; c'est l'OLED qui coûte

Travail à la maison (45 min)

Réécris `alerte` en classe **C++** (constructeur prenant broche + hystérésis, méthode `tick()`) au lieu de fonctions + `static` . Compare les deux styles dans ton journal : qu'est-ce que la classe apporte ici ?
(réponse attendue : plusieurs instances possibles, état impossible à corrompre de l'extérieur — cf. TD 02.)

→ Fiche suivante : [Séance 3 — FSM d'interface et journalisation](#)